Ensuring Quality in Software Projects

Series of lectures by Yegor Bugayenko

Abstract:
The course is a series of loosely coupled pieces of advice related to quality of software development. Pragmatic programmers may listen to them if they don't want to tolerate chaos in their projects. The course is not only about coding practices, but also about static analysis, test coverage, bug tracking, dependency and artifact management, build automation, DevOps, and many other things. If we don't do them right, they may severely jeopardize the quality of the entire project, no matter how good are your algorithms.

What is the goal?

The primary objective of the course is to explain by example how quality in a software project can be put under control.

Who is the teacher?

I'm developing software for more than 30 years, being a hands-on programmer (see my GitHub account: @yegor256) and a manager of other programmers. At the moment I'm a director of an R&D laboratory in Huawei. Our primary research focus is software quality problems. You may find some lectures I've presented at some software conferences on my YouTube channel. I also published a few books and wrote a blog about software engineering and object-oriented programming.

Why this course?

In one of my videos a few years ago I explained what I believe is killing most software projects: it's the chaos they can't control. Most of us programmers start projects full of enthusiasm and best intentions. We are confident that this time the design will be solid, the code will be clean, and our customers will be happy because there will be no bugs. Eventually, sooner rather than later, the reality appears to be as bad as it was in the previous project: the code is messy, the design resembles spaghetti, and the bugs are unpredictable and hard to fix. We learn some lessons, abandon the project, and start a new one, again with the best intentions. But in a new project nothing changes. Most programmers that I know run in this cycle for decades. I believe, this course may help you not become one of them.

What's the methodology?

The course is a collection of individual cases not closely connected to each other. Each lecture discusses a single open-source GitHub repository. Each discussion highlights technical decisions made in the repository and explains them in details. Each lecture ends with a conclusion and a formulated recommendation. The recommendations may help students prevent and control chaos in their own future projects.

Course Structure

Prerequisites to the course (it is expected that a student knows this):

- How to use Git
- How to code
- How to design software
- How to write automated tests
- How to deploy

After the course a student *hopefully* will understand:

- How to avoid code smells
- How to convince a manager that quality is important
- How to deal with negligence of other programmers
- How to hire a programmer who cares
- How to argue with customers about quality
- How to discipline fellow programmers
- How to protect yourself from chaos

Also, a student will be able to practice:

- Source control: Git, Subversion
- Build automation: Make, Maven, Gradle, Grunt, Rake
- Dependencies: Maven Central, NpmJS, RubyGems, PyPi
- Static analysis: Clang-Tidy, SpotBugs, Coverity
- Style checking: Checkstyle, PMD, Rubocop, Eslint, Qulice
- Automated tests: JUnit, Mocha
- Integration tests: Cucumber, Selenium, Cross-Browser
- Performance testing: JMeter
- Mocking frameworks: Mockito, PowerMock
- Textual documentation: Markdown, Wiki, LaTeX, CNL
- Bug tracking: GitHub, JIRA, Bugzilla
- Code reviews: GitHub, Gerrit, Crucible
- Test coverage: JaCoCo, Codecov
- Mutation coverage: PIT
- Property-based testing: Quickcheck
- DevOps: Docker, Heroku, AWS
- Pre-flight builds: GitHub Actions, Jenkins, Rultor
- Metrics: SonarQube, CodeClimate, jPeek.

Lectures

This is a list of cases that will be discussed at the lectures:

1. Ruby style checking with Rubocop in yegor256/sibit
2. XML style checking in yegor256/xcop
3. Custom Checkstyle rule in yegor256/qulice
4. GitHub actions in yegor256/fibonacci and jcabi/jcabi-xml
5. Code reviews in objectionary/eo
6. Making and testing a new GitHub Action in yegor256/latexmk-action
7. Npm dependencies and Grunt in objectionary/eoc
8. Managing Maven dependencies in yegor256/takes (with Renovate)
9. Multi-module `pom.xml` in objectionary/eo
10. Build automation with `Makefile` in yegor256/fibonacci
11. Build automation with Gradle in objectionary/eo-intellij-plugin
12. Making a Ruby gem in yegor256/cobench
13. Packaging for CTAN in yegor256/iexec
14. Deploying to Maven Central in yegor256/cactoos
15. Docker image releasing to the Hub in yegor256/rultor-image
16. Deploying Java app to Heroku in yegor256/rultor
17. Deploying Java app to Dokku in yegor256/jare
18. Deploying a Ruby web app in yegor256/sixnines
19. Reversive deployment to AWS EC2 in yegor256/s3auth
20. Unit vs. Integration testing in jcabi/jcabi-xml
21. Fake GitHub and AWS S3 objects in jcabi/jcabi-github and jcabi/jcabi-s3
22. Testing for thread-safety in yegor256/cactoos
23. Integration testing against DynamoDB Local in yegor256/rultor
24. Testcontainers in yegor256/threecopies
25. Parametrized testing with YAML in objectionary/eo
26. Credentials testing in yegor256/0pdd
27. Headless in-browser testing with Selenium in yegor256/jare
28. Stress testing in yegor256/takes
29. BDD with Cucumber in cqfn/pdd
30. JaCoCo coverage control in yegor256/cactoos
31. CNL for requirements specification in yegor256/requs
32. Hits-of-code and other metrics in yegor256/cobench and yegor256/hoc
33. Calculating Java cohesion metrics in cqfn/jpeek
34. Developers performance in yegor256/cobench
35. Documenting `README.md` of a Java library in yegor256/takes

Students are welcome to pick most interesting cases studies.

Laboratory Classes

A few following laboratory classes may support the course, where students will be asked to solve some of these tasks (the most complex are at the bottom):

1. Configure GitHub Action to publish code coverage to codecov
2. Make both JUnit4 and JUnit5 tests work inside one Java repository
3. Create a new unit test in BDD style, using Cucumber or a similar framework
4. Design a coding style guide (in Markdown) for your favorite language
5. Configure GitHub Actions to deploy a new JavaScript library to npmjs
6. Automate headless Selenium+Safari integration testing in a simple web app
7. Automate mutation coverage publishing to GitHub Pages
8. Configure build to fail if test coverage is lower than 80%
9. Create a new plugin for GitHub Actions to validate the layout of repository
10. Merge a pull request improving coding style to a 10k+ GitHub library
11. Create an automated test for Java+Hibernate+MySQL with testcontainers
12. Create GitHub Action to spell check `README.md` using aspell
13. Install Jenkins and configure it to merge branches on demand
14. Configure GitHub Action to send a message to Telegram when the build fails
15. Automate cross-browsing testing of a web app using BrowserStack
16. Configure property-based testing in an existing repository
17. In a existing Java project automated with Maven, replace Maven with Makefile
18. Automate comparison of two static analyzers, find out which one is stronger
19. Develop a Maven plugin to check the quality of POM file
20. Create a new style checking rule for PMD, to prohibit the use of non-final classes
21. Using aspell, create a style checker validating grammar in Java comments

There could be other tasks too.

Grading

Students may form groups of up to four people. Each group will present their own public GitHub repository with a software module inside. The group will make a presentation of the quality control mechanisms that are present in the repository. They will have to explain during a 10-minutes oral presentation with live GitHub demonstration via screen sharing:

- How enabled quality ensuring mechanisms work?
- Why such mechanisms are in use?
- How they help ensure quality?
- How often they get activated?
- What are the drawbacks of them?
- What mechanism are not used and why?

Most probably, there will be no more than 20% of "A" marks, no more than 40% of "B," and the rest will go to "C" and "D." However, this distribution is not mandatory: if all students make excellent presentations, everybody will get "A."

Attendance will be tracked at the lectures. If you attend more than 75% of all lectures, you will not get less than "C".

At the laboratory classes each group will have to complete three home works and defend them verbally on-site. A completion of less than two will give everybody in the group a negative point, a completion of three — will give a positive point; the point will be added to the grade given by the lecturer.

Higher grades will be given for:

- Better understanding of the reasons behind used mechanisms,
- How they help ensure quality,
- How often they get activated, and
- What are the drawbacks of them.

A retake exam is possible, following exactly the same procedure. However, the highest mark most probably possible at the retake is "C."

Students are highly advised to discuss their repositories and quality ensuring mechanisms with each other, before the final exam, in order to understand their relative positions and maybe trigger new ideas.

Learning Material

The following books are highly recommended to read (in no particular order):

Steve McConnell, *Software Estimation: Demystifying the Black Art*

Robert Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*

Steve McConnell, *Code Complete*

Frederick Brooks Jr., *Mythical Man-Month, The: Essays on Software Engineering*

David Thomas et al., *The Pragmatic Programmer: Your Journey To Mastery*

Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*

David West, *Object Thinking*

Yegor Bugayenko, *Code Ahead*

Michael Feathers, *Working Effectively with Legacy Code*

Jez Humble et al., *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*

Michael T. Nygard, *Release It!: Design and Deploy Production-Ready Software*